

توضیحات کد پانداس

قسمت اول

در این قسمت ابتدا با استفاده از `pd.read_csv` فایل های csv را خوانده. سپس با استفاده از `DataFrame.info()` و `DataFrame.describe()` اطلاعات کلی از دیتافریم ها بدست می آوریم.

قسمت دوم

در این قسمت با ترکیب کردن چند دیتافریم، به دیتافریم مناسبی دست میابیم که به کمک آن میتوانیم به همه اطلاعات مورد نیاز دسترسی داشته باشیم.

```
In [ ]: merged = pd.merge(players[['id_player', 'id_team']], goals[['id_player', "goals"]])
merged1 = pd.merge(merged, goalkeeping[['id_player", "goals_conceded"]])
merged2 = pd.merge(merged1, teams[["team_id", "team"]], left_on="id_team", right_on="team_id")
```

در مرحله دیتافریم بدست آمده را به کمک `DataFrame.groupby()` دسته بندی میکنیم.

```
In [ ]: grouped = merged2.groupby('id_team')
```

با جمع محتواهای ستون های `goals` و `goals_conceded` از سطرهایی که متعلق به یک تیم هستند. گل های زده و گل های خورده هر تیم را میتوانیم محاسبه کنیم.

```
In [ ]: chart = grouped[['goals', 'goals_conceded']].sum()
chart
```

دیتافریم بالا شامل 35 سطر است که نشان دهنده آن است که اطلاعات بازیکنان یک تیم درون دیتاست ورودی ما وجود ندارد؛ لذا با استفاده از بلاک کد زیر اسم و کد آن تیم را بدست آوردم.

```
In [ ]: list1 = teams['team_id'].values
for i in list1:
    if i not in chart.index:
        not_valid = i
```

```
row = teams[teams['team_id'] == i]
not_valid_index = row['team'].index
print("name of the club that its data hasn't registered: ", end="")
print(row['team'].values[0])
print(f'this is team_id: {i}')
```

در این قسمت با استفاده از `DataFrame.sort_values` و دادن پارامترهای مناسب، دیتافریم را براساس گل های زده مرتب میکنیم. پارامتر اول این تابع، لیست ستون هایی است که بر اساس آن عمل مرتب سازی را انجام میدهد و پارامتر دوم آن صعودی یا نزولی بودن اهمیت ستون ها را مشخص میکند.

```
In [ ]: # Sorting teams based on their goals datas
output = chart.sort_values(['goal_difference', 'goals'], ascending=[False, True])
output
```

نمایش بصری

در این قسمت به نمایش بصری داده های دیتافریم نهایی پرداخته شده. نمایش حاوی سه نمودار "گل های زده" و "گل های خورده" و "تفاضل گل" است.

```
In [ ]: # visualization
import matplotlib.pyplot as plt

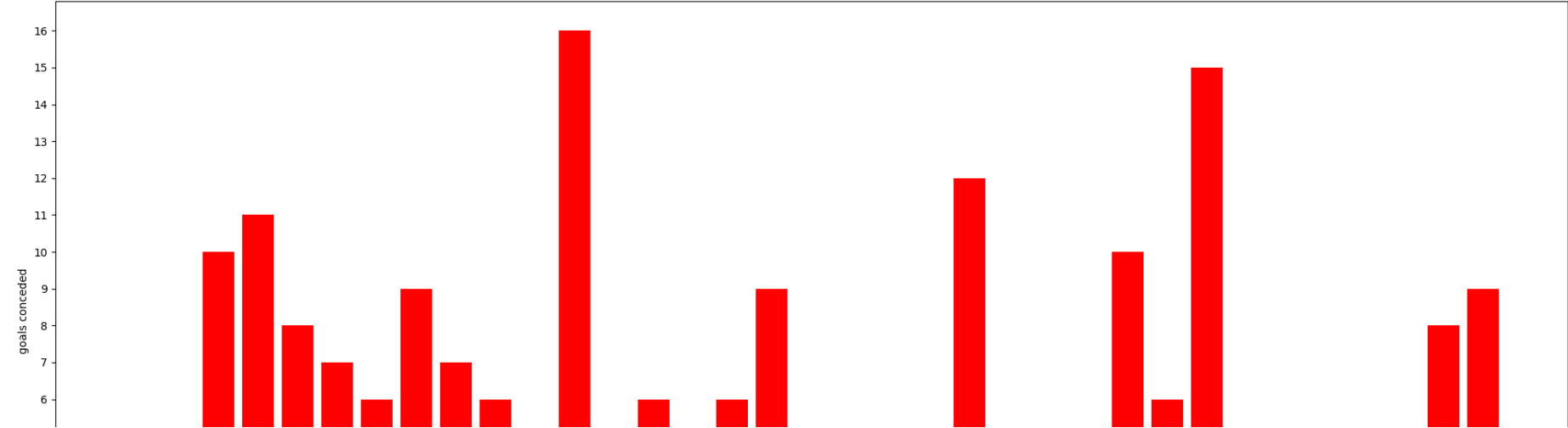
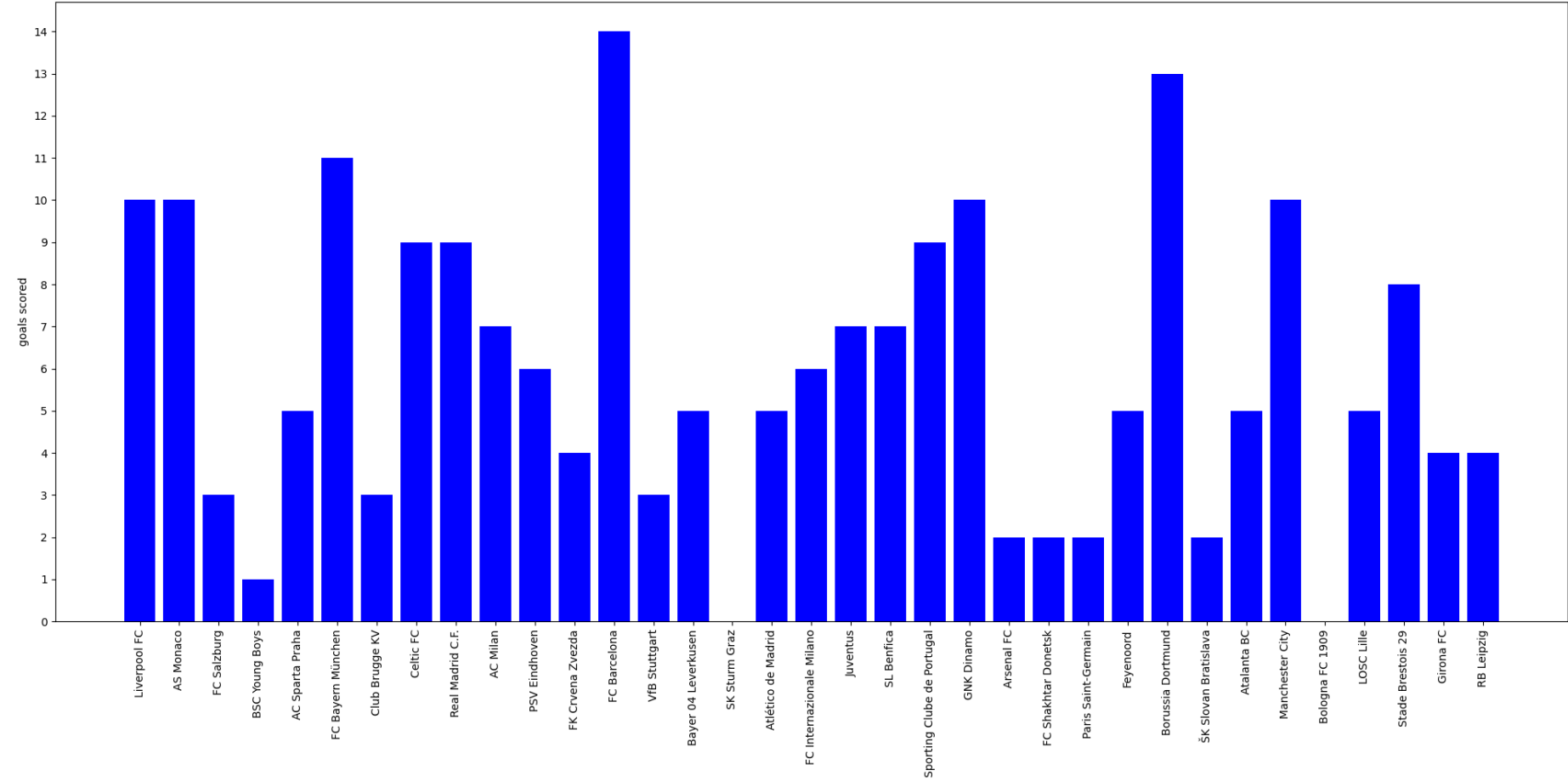
team_name = update_teams['team'].values
goals_scored = chart['goals'].values
goals_conceded = chart['goals_conceded'].values
goal_difference = chart['goal_difference'].values

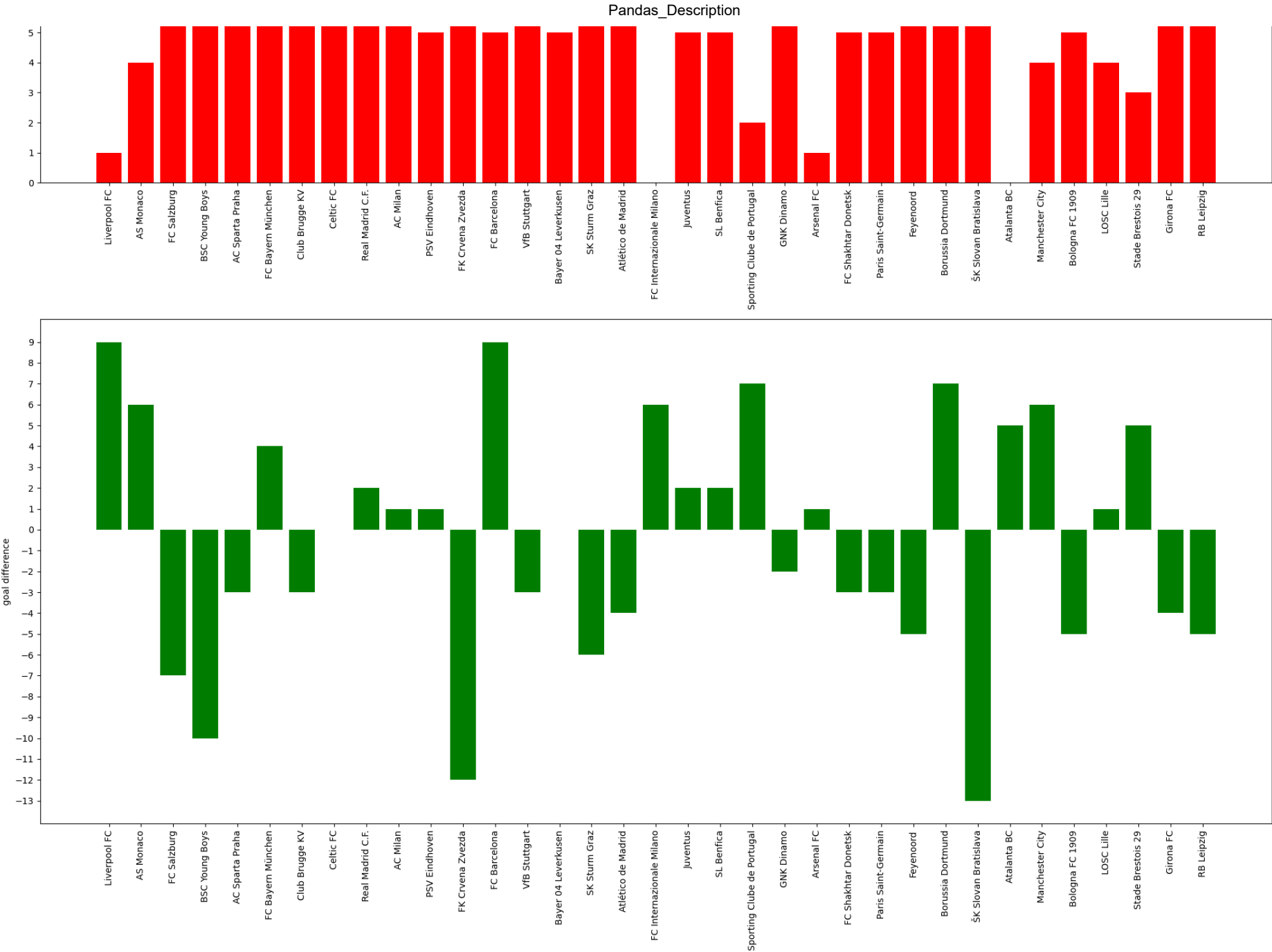
fig, ax = plt.subplots(3, 1, figsize=[20,30])
ax[0].bar(team_name, goals_scored, color='blue')
ax[0].tick_params(axis='x', labelrotation=90, labelcolor='black')
ax[0].set_yticks(np.arange(0, max(goals_scored)+1, 1))
ax[0].set_ylabel("goals scored")

ax[1].bar(team_name, goals_conceded, color='red')
ax[1].tick_params(axis='x', labelrotation=90, labelcolor='black')
ax[1].set_yticks(np.arange(0, max(goals_conceded)+1, 1))
ax[1].set_ylabel("goals conceded")

ax[2].bar(team_name, goal_difference, color='green')
ax[2].tick_params(axis='x', labelrotation=90, labelcolor='black')
ax[2].set_yticks(np.arange(min(goal_difference), max(goal_difference)+1, 1));
```

```
ax[2].set_ylabel("goal difference")  
fig.tight_layout()
```





قسمت سوم

در این قسمت از ما خواسته شده تا تمامی دیتافریم های موجود را با هم ترکیب کنیم. در تمامی دیتافریم های موجود به جز `teams` ستون `id_player` مشترک است. همچنین بین دیتافریم های `teams` و `players` نیز ستون `id_team` مشترک است. برای ترکیب دیتافریم ها و جلوگیری از استفاده حلقه، از تابع `reduce` از کتابخانه `functools` استفاده شده است.

```
In [ ]: from functools import reduce

merged3 = pd.merge(players, teams, left_on='id_team', right_on='team_id')
datas = [merged3, attacking, attempts, defending, disciplinary, distribution, goalkeeping, goals, key_stats]

merged4 = reduce(lambda left, right : pd.merge(left, right), datas)
merged4
```

در نهایت با استفاده از تابع `set_index()` طبق خواسته مسئله، دیتافریم ایندکس گذاری میشود.

```
In [ ]: merged4.set_index(['team', 'player_name'])
```

قسمت چهارم

در ابتدا دو دیتافریم `attempts` و `players` را برای دسترسی به اطلاعات مورد نظر ترکیب میکنیم.

```
In [ ]: # merging dataframes to access players' data and attempts' data simultaneously
attempts1 = pd.merge(attempts, players)
attempts1
```

در مرحله بعد با ایجاد ستون جدید `accuracy`، دقت هر بازیکن در موقعیت های گل را تعیین میکنیم.

```
In [ ]: attempts1['accuracy'] = attempts1['attempts_on_target'] / attempts1['total_attempts']
attempts1
```

در اینجا دیتافریم جدیدی از دیتافریم قبلی کپی میکنیم که ستون `accuracy` آن حاوی NaN نباشد.

```
In [ ]: attempts2 = attempts1[attempts1['accuracy'].notna()]
attempts2
```

با استفاده از `quantile(x)` مقدار چارک ها را محاسبه میکنیم.

```
In [ ]: q1 = attempts2['accuracy'].quantile(0.25)
        q2 = attempts2['accuracy'].quantile(0.5)
        q3 = attempts2['accuracy'].quantile(0.75)
```

در نهایت با بازه بندی مقادیر `accuracy` ، و استفاده از تابع `mode()` ، بیشترین پست در هر بازه را تعیین میکنیم.

قسمت پنجم

بخش اول

در بخش اول مجددا دیتافریم ها را به کمک `pd.read_csv` میخوانیم و با استفاده از `set_index()` ایندکس گذاری میکنیم و سپس با استفاده از `dropna` و متود `how='all'` ، سطرهایی که تمام مقادیر آن NaN باشد را حذف میکند.

```
In [ ]: attempts = pd.read_csv('./dataset/attempts_data.csv')
        attempts = pd.DataFrame(attempts)
        attempts.set_index('id_player', inplace=True)
        attempts.dropna(how='all', inplace=True)
```

این کار را به ترتیب برای تمامی دیتافریم ها تکرار میکنیم.

بخش دوم

ابتدا مجددا دیتافریم `players` را میخوانیم. سپس مقادیر یونیک ستون `position` را با استفاده از تابع `unique()` بدست میآوریم.

```
In [ ]: players = pd.read_csv('./dataset/players_data.csv')
        players = pd.DataFrame(players)
        players['position'].unique()
        # positions = players['position'].values

array(['STRIKER', nan, 'UNKNOWN', 'CENTRAL_MIDFIELDER', 'CENTRE_BACK',
       'ATTACKING_MIDFIELDER', 'FULL_BACK', 'WINGER', 'GOALKEEPER',
       'DEFENSIVE_MIDFIELDER'], dtype=object)
```

پیدا کردن دروازه بان ها

در این قسمت، ان دسته از NaN مقدارهایی که `field_position` آن ها برابر "Goalkeeper" بوده است، مشخصا، مقدار `position` آن ها نیز باید "GOALKEEPER" باشد.

```
In [ ]: nan_index_goalkeeper = players.iloc[nan_index][players['field_position'] == 'Goalkeeper'].index
```

```
In [ ]: update_players = players.copy()
```

```
In [ ]: update_players.iloc[nan_index_goalkeeper] = 'GOALKEEPER'
```

پیدا کردن مدافعین

در این قسمت پس از تست و بررسی چند فیچر، به این نتیجه رسیدم که ستون `crosses_attempted` رابطه نزدیک تری با تعیین مدافعان دارد. به این شکل که اگر مقدار ان از میانگین داده ها بیشتر باشد، احتمالا `position` ان متعلق به پست "FULL_BACK" خواهد بود و اگر کمتر از میانگین باشد احتمالا متعلق به پست "CENTRE_BACK" خواهد بود.

آن دسته از NaN مقدارهایی که ستون `crosses_attempted` ان نیز NaN بوده است را برابر "UNKNOWN" قرار دادم.

مقدار میانگین که بر اساس ان نوع مدافع تعیین شده است:

```
In [ ]: train_set = df.drop(nan_index_defender)
defender = train_set[train_set["field_position"] == 'Defender']
threshold = defender["crosses_attempted"].mean()
threshold
```

2.9844559585492227

```
In [ ]: full_back = df.iloc[nan_index_defender][df["crosses_attempted"] >= threshold].index
df.loc[full_back, 'position'] = "FULL_BACK"
```

```
In [ ]: centre_back = df.iloc[nan_index_defender][df['crosses_attempted'] < threshold].index
df.loc[centre_back, 'position'] = "CENTRE_BACK"
```

داده هایی که ستون `crosses_attempted` آن نیز برابر NaN بود را برابر "UNKNOWN" قرار میدهم.

```
In [ ]: unknown = df.loc[nan_index_defender][df['crosses_attempted'].isnull()].index
```



```
df.loc[unknown, 'position'] = "UNKNOWN"
```

در ادامه رابطه مناسبی برای تعیین پست هافبک ها و مهاجمین پیدا نکردم: